

Modélisation avancée des Systèmes d'Information

07 - Decorator

Mohamed ZAYANI

2025/2026

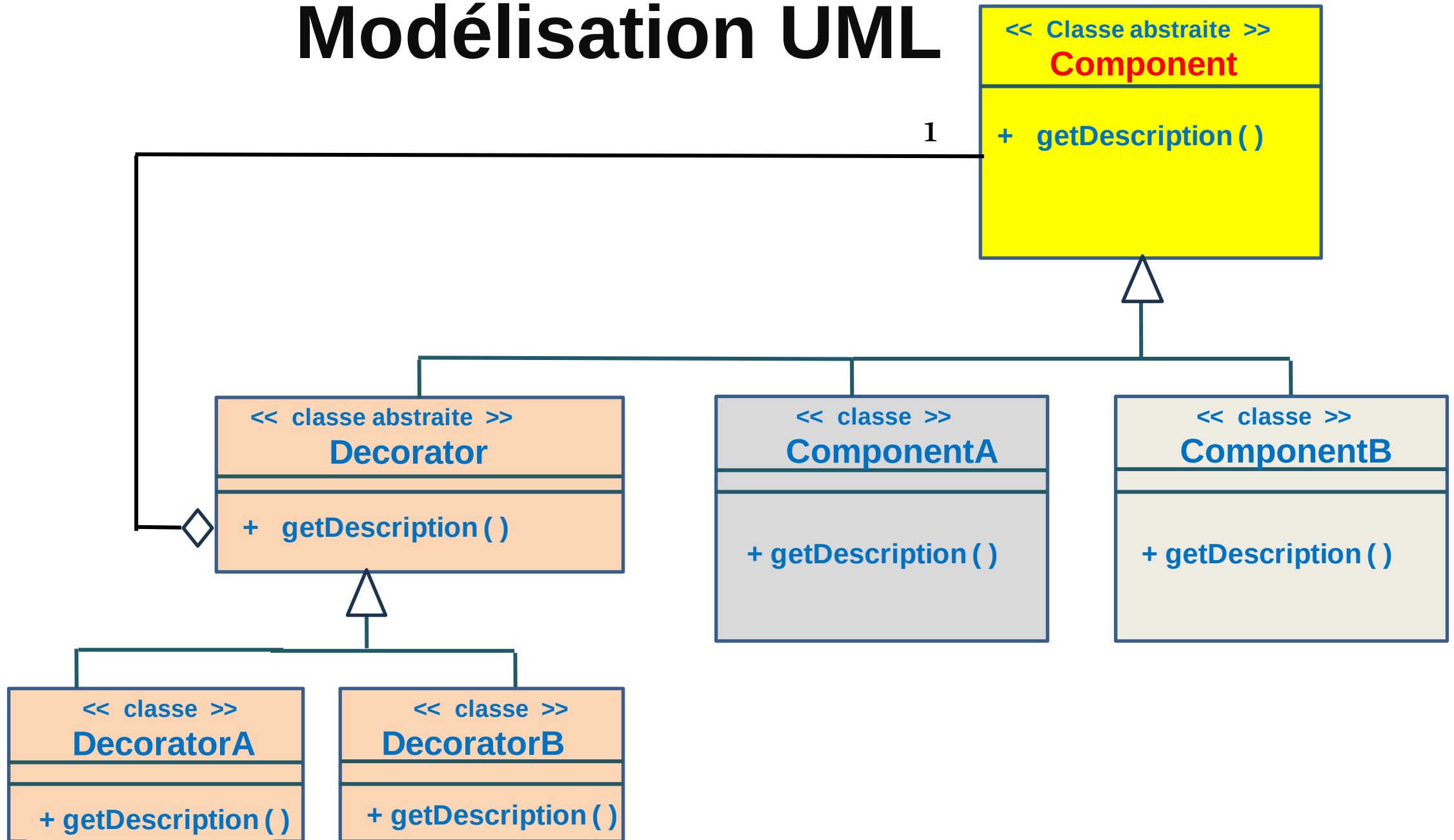
Définition du Decorator

- Le patron de conception **Decorator** est un **patron structurel** qui permet d'ajouter **dynamiquement** des fonctionnalités à un objet sans modifier son code.
- Il encapsule un objet existant dans une nouvelle classe (**le décorateur**) qui enrichit son comportement.
- **Objectif principal:**
 - ❖ Ajouter des fonctionnalités de manière flexible et évolutive.
- **Problématique:**
 - Comment enrichir un objet avec de nouvelles fonctionnalités sans modifier sa classe ?
 - L'héritage ne permet pas de combiner dynamiquement des comportements.
 - Étendre les fonctionnalités d'un objet **sans hériter** de sa classe.
- **Exemple:**
 - Dans un logiciel de traitement de texte, un texte peut être:
 - ❖ Simple
 - ❖ Gras
 - ❖ Souligné
 - ❖ Encadré
 - ❖ Créer une sous-classe pour chaque combinaison (Gras + Souligné + Encadré) devient **ingérable**.

Solution proposée par Decorator

- Définir une **interface commune** ou une **classe abstraite** pour les composants de base.
- Créer des **décorateurs** qui implémentent cette interface et ajoutent des fonctionnalités.
- **Composer** les décorateurs pour enrichir progressivement l'objet.
- **Principe:**
 - Étendre les fonctionnalités d'un objet sans hériter de sa classe.
- **Avantages :**
 - **Flexibilité** : Ajouter ou retirer des fonctionnalités dynamiquement.
 - **Respect** des principes SOLID (Open/Closed).
 - **Combinaison libre** des fonctionnalités.

Modélisation UML



Implémentation JAVA

```
//structure de base
public abstract class Boisson
{
    abstract String getDescription();
    abstract double getPrix();
}
```

```
//Composant concret : Café simple
public class Cafe extends Boisson
{
    @Override
    public String getDescription()
    {
        return "Café";
    }

    @Override
    public double getPrix()
    {
        return 1.5;
    }
}
```

```
//Décorateur abstrait
public abstract class BoissonDecorator extends Boisson
{
    protected Boisson boisson;

    public BoissonDecorator(Boisson boisson)
    {
        this.boisson = boisson;
    }
    @Override
    public String getDescription()
    {
        return boisson.getDescription();
    }
    @Override
    public double getPrix()
    {
        return boisson.getPrix();
    }
}
```

Implémentation JAVA

```
//Décorateur1 : Ajouter du lait
public class Lait extends BoissonDecorator {
    public Lait(Boisson boisson) {
        super(boisson);
    }
    @Override
    public String getDescription() {
        return boisson.getDescription() + ", Lait";
    }
    @Override
    public double getPrix() {
        return boisson.getPrix() + 0.5;
    }
}
```

```
//Décorateur2 : Ajouter du chocolat
public class Chocolat extends BoissonDecorator {
    public Chocolat(Boisson boisson) {
        super(boisson);
    }
    @Override
    public String getDescription() {
        return boisson.getDescription() + ", Chocolat";
    }
    @Override
    public double getPrix() {
        return boisson.getPrix() + 0.7;
    }
}
```

```
public class Client {
    public static void main(String[] args) {
        // Boisson de base : Café
        Boisson boisson = new Cafe();
        System.out.println(boisson.getDescription() + " => "
            + boisson.getPrix() + "DT");

        // Café avec lait
        boisson = new Lait(boisson);
        System.out.println(boisson.getDescription() + " => "
            + boisson.getPrix() + "DT");

        // Café avec lait et chocolat
        boisson = new Chocolat(boisson);
        System.out.println(boisson.getDescription() + " => "
            + boisson.getPrix() + "DT");
    }
}
```

Résultat:

```
Café => 1.5DT
Café, Lait => 2.0DT
Café, Lait, Chocolat => 2.7DT
```

Cas d'utilisation du Patron Decorator

- **Interfaces graphiques (GUI)** : Ajouter des bordures, des couleurs, des ombres aux composants.
- **Systèmes de fichiers** : Ajouter des couches de compression ou de chiffrement.
- **Jeux vidéo** : Ajouter des équipements ou des pouvoirs spéciaux aux personnages.
- **Flux d'I/O en Java** : `BufferedInputStream`, `DataInputStream` décorent `InputStream`.